/login/oauth2/code/{registrationId}. For Edge Service, the full URL is http://localhost:9000/login/oauth2/code/keycloak, which we already configured in Keycloak as a valid redirect URL.

Scopes are an OAuth2 concept for limiting an application's access to user resources. You can think of them as roles, but for applications instead of users. When we use the OpenID Connect extension on top of OAuth2 to verify the user's identity, we need to include the `openid` scope to inform the Authorization Server and receive an ID Token containing data about the user authentication. The next chapter will explain more about scopes in the context of authorization.

Now that we've defined the integration with Keycloak, let's configure Spring Security to apply the desired security policies.

11.4.3 Basic Spring Security configuration

The central place for defining and configuring security policies in Spring Security is a `SecurityWebFilterChain` class. Edge Service is currently configured to require user authentication for all endpoints, and it uses an authentication strategy based on a login form. Let's change that to use OIDC authentication instead.

The `ServerHttpSecurity` object provides two ways of configuring an OAuth2 Client in Spring Security. With `oauth2Login()`, you can configure an application to act as an OAuth2 Client and also authenticate users through OpenID Connect. With `oauth2Client()`, the application will not authenticate users, so it's up to you to define another authentication mechanism. We want to use OIDC authentication, so we'll use `oauth2Login()` and the default configuration. Update the `SecurityConfig` class as follows.